



JAVA OOPS CONCEPT!!

Oops stand for Object-Oriented Programming language, the main purpose of oops is to deal with real world entity using programming language.

1. What are the four main principles of Object-Oriented Programming? Explain each with an example.

Oops four pillars are :--

➤ **Abstraction** :-- Abstraction in Java is one of the fundamental OOP concepts that helps in hiding unnecessary details and only showing the essential features of an object.

- Declare by using “abstract” keyword.
- Example of abstract concept:--

```
abstract class Bike{ 1 usage 1 inheritor
    abstract void run(); 1 usage 1 implementation
}
class Honda extends Bike{ 2 usages
public void run () 1 usage
{
    System.out.println("Honda bike is running.");
}
}

public class abstract_concept {
    public static void main(String[] args) {
        Honda h=new Honda();
        h.run();
    }
}
```

➤ **Encapsulation** :-- Encapsulation in Java is one of the core principle of OOPs. It is defined as the wrapping of data in a single unit.

- It restricts direct access to the class's fields.

- Benefits of Encapsulation :-

1. Data hiding.
2. Security.
3. Better code maintainability.
4. Flexibility.

- Example of encapsulation concept:--

```
public class Student {  
    private String name; 3 usages  
    private int age; 3 usages  
    public Student(String name, int age) { 1 usage  
        this.name = name;  
        this.age = age;  
    }  
    public String getName() { 1 usage  
        return name;  
    }  
    public void setName(String name) { no usages  
        this.name = name;  
    }  
    public int getAge() { 1 usage  
        return age;  
    }  
    public void setAge(int age) { no usages  
        if (age > 0) {  
            this.age = age;  
        }  
    }  
    public static void main(String[] args) {  
        Student student = new Student( name: "Kushagra", age: 18);  
        System.out.println("Name: " + student.getName());  
        System.out.println("Age: " + student.getAge());  
    }  
}
```

➤ **Inheritance** :-- Inheritance is a fundamental concept in OOPs that allows a class to inherit properties and behaviours from another class.

- Superclass and Subclass :- The class that provides properties and behaviours is called the superclass, and the class that inherit them is called the subclass.
- Types of Inheritance :-
 1. Single Inheritance
 2. Multilevel Inheritance
 3. Hierarchical Inheritance
- Benefits of Inheritance :-
 1. Promotes code reuse.
 2. Enhances maintainability.
 3. Establishes relationship between classes.
- Example of inheritance concept :-

```
class Animal { 1 usage 1 inheritor
    String name; 3 usages

    public Animal(String name) { 1 usage
        this.name = name;
    }

    public void eat() { 1 usage
        System.out.println(name + " is eating.");
    }
}

class Dog extends Animal { 2 usages
    String breed; 1 usage
    public Dog(String name, String breed) { 1 usage
        super(name);
        this.breed = breed;
    }

    public void bark() { 1 usage
        System.out.println(name + " is barking.");
    }
}

public class Main {
    public static void main(String[] args) {
        Dog myDog = new Dog( name: "Buddy", breed: "Golden Retriever");
        myDog.eat();
        myDog.bark();
    }
}
```

➤ Polymorphism :-- Polymorphism is a fundamental concept in OOPs that allows a single interface to be used for different types of objects.

- Key Aspects of Polymorphism :-

1. Compile-time Polymorphism (Method Overloading).
2. Runtime Polymorphism (Method Overriding).
3. Dynamic Method dispatch.

- Advantages of Polymorphism:-

1. Increase code flexibility.
2. Promotes code reusability.
3. Enhances maintainability.

- Example of polymorphism concept :-

```
class Polygon { 2 usages 2 inheritors
    public void render() { 2 usages 2 overrides
        System.out.println("Rendering Polygon...");
    }
}

class Square extends Polygon { 2 usages
    public void render() { 2 usages
        System.out.println("Rendering Square...");
    }
}

class Circle extends Polygon { 2 usages

    // renders circle
    public void render() { 2 usages
        System.out.println("Rendering Circle...");
    }
}

class Main {
    public static void main(String[] args) {
        Square s1 = new Square();
        s1.render();
        Circle c1 = new Circle();
        c1.render();
    }
}
```

2. How does inheritance work in OOP? Write a sample program to demonstrate single and multilevel inheritance.

Inheritance in OOPs, is a mechanism that allows a class (subclass/child class) to acquire properties and behaviour of another class (superclass/parent class).

It enables code reuse and establishes a relationship between classes.

Sample program of inheritance:-

```
class Animal { 1 usage 2 inheritors
    void eat() { 1 usage
        System.out.println("This animal eats food.");
    }
}
class Mammal extends Animal { 1 usage 1 inheritor
    void walk() { 1 usage
        System.out.println("This mammal walks on land.");
    }
}
class Dog extends Mammal { 2 usages
    void bark() { 1 usage
        System.out.println("The dog barks.");
    }
}
public class Main {
    public static void main(String[] args) {
        Dog myDog = new Dog();
        myDog.eat();
        myDog.walk();
        myDog.bark();
    }
}
```

3. What is the difference between method overloading and method overriding? Provide code examples.

Method overloading and method overriding are both forms of polymorphism in Java, but they serve different purposes.

Overloading is compile-time polymorphism, providing multiple versions of a method.

Overriding is runtime polymorphism, allowing a subclass to modify inherited behaviour.

Example of overloading:-

```
class MathOperations { 2 usages
    int multiply(int a, int b) { 1 usage
        return a * b;
    }
    double multiply(double a, double b) { 1 usage
        return a * b;
    }
}

public class Main {
    public static void main(String[] args) {
        MathOperations math = new MathOperations();
        System.out.println(math.multiply(a: 5, b: 3));
        System.out.println(math.multiply(a: 5.5, b: 3.2));
    }
}
```

Example of overriding:-

```

class Animal { 2 usages 1 inheritor
    void makeSound() { 1 usage 1 override
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal { 1 usage
    @Override 1 usage
    void makeSound() {
        System.out.println("Dog barks");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal myAnimal = new Dog();
        myAnimal.makeSound();
    }
}

```

4. What is encapsulation and how does it help in software development? Show how it is implemented in code.

Encapsulation is one of the core principles of OOPs. It refers to the bundling of data and methods that operate on that data into a single unit, often a class, and restricting direct access to the data by using access modifiers.

Encapsulation helps in software development by :-

1. Protecting Data.
2. Maintaining Data Integrity.
3. Improving Code Maintainability.
4. Enhancing Modularity.
5. Encapsulation support Abstraction.

```
class BankAccount { 2 usages
    private String accountHolder; 1 usage
    private double balance; 5 usages
    public BankAccount(String accountHolder, double initialBalance) { 1 usage
        this.accountHolder = accountHolder;
        this.balance = initialBalance;}
    public double getBalance() { 2 usages
        return balance;}

    public void deposit(double amount) { 1 usage
        if (amount > 0) {
            balance += amount;
            System.out.println("Deposited: $" + amount);
        } else {
            System.out.println("Deposit amount must be positive.");
        }
    }

    public void withdraw(double amount) { 1 usage
        if (amount > 0 && amount <= balance) {
            balance -= amount;
            System.out.println("Withdrawn: $" + amount);
        } else {
            System.out.println("Insufficient funds or invalid amount.");
        }
    }
}

public class Main {
    public static void main(String[] args) {
        BankAccount myAccount = new BankAccount( accountHolder: "Alice", initialBalance: 1000);
        System.out.println("Balance: $" + myAccount.getBalance());
        myAccount.deposit( amount: 500);
        myAccount.withdraw( amount: 300);
        System.out.println("Final Balance: $" + myAccount.getBalance());
    }
}
```